

Backend Portfolio

Petory

문제를 재현하고 측정하며 개선하는 백엔드 개발자
박영범

Spring Boot, JPA, MySQL 기반 반려동물 통합 플랫폼.
N+1, 동시성, 위치 검색, NLP 호출 정책을 실제 사용자 흐름 기준으로
재현하고 개선했습니다.

Spring Boot

JPA

MySQL

Redis

WebSocket

FastAPI

React

Portfolio site: makkong1.github.io/makkong1-github.io

Backend repo: github.com/makkong1/Petory

포지셔닝

문제 해결 방향

기능 구현 자체보다, 기능이 실제 사용량과 동시 접근을 만났을 때 어떤 병목과 데이터 불일치가 생기는지 확인했습니다. 쿼리 수, 실행 시간, 메모리 사용량을 기준으로 Before/After를 비교하고, 같은 패턴을 여러 도메인에 재사용했습니다.

핵심 강점

JPA 연관관계와 DTO 변환 과정에서 생기는 N+1을 로그와 시나리오로 추적하고, IN 절 배치 조회, Fetch Join, Map 매핑으로 해결했습니다.

설계 태도

비관적 락, 원자적 UPDATE, DB Unique 제약조건을 문제 성격별로 구분했습니다. 정합성이 중요한 기능과 부가 기능의 실패 정책도 분리했습니다.

검증 방식

문서와 데모를 함께 두어 문제 상황, 원인, 수정 전후 흐름, 측정 결과를 같은 저장소에서 추적할 수 있게 정리했습니다.

99.8%

Care 쿼리 감소

2400개에서
4-5개

24.8x

Board 속도 개선

745ms에서
30ms

4개

Chat 목록 쿼리

채팅방 수와 무관

프로젝트 개요

User

JWT, OAuth2,
이메일 인증, 반려동물,
제재

Board

게시판, 댓글, 반응,
인기글 스냅샷

Care

요청/지원, 채팅 매칭,
거래 확정, 리뷰

Location

공공데이터, 지도,
카테고리/반경 검색

Recommendation

이벤트 기반 intent
signal, NLP, 추천
카드

Meetup

모임 생성/참여, 위치
검색, 참여자 제한

Chat

WebSocket/STOMP,
읽음 상태, 최신 메시지

Common

Payment,
Notification,
Report, Admin,
File

Frontend

React 19, Vite, Styled Components,
Recharts, Mermaid, Axios, mock 기반
라이브 데모

Backend

Spring Boot 3.x, Java 17, Spring
Data JPA, MySQL, Redis, Spring
Security, WebSocket, SSE/FCM

시스템 아키텍처



레이어드 아키텍처

Controller는 요청/응답과 검증, Service는 비즈니스 로직과 트랜잭션, Repository는 데이터 접근, Entity는 도메인 모델을 담당합니다.

도메인 중심 구조

도메인별
controller/service/repository/entity/dto/converter
구조를 유지해 기능 경계를 읽기 쉽게 만들었습니다.

외부 연동

Naver Map, OAuth2,
SMTP, FCM, FastAPI NLP
서버를 Spring Boot 뒤에 두어
프론트가 직접 외부 분석 서버를 호출하지
않게 했습니다.

데이터 모델과 문서화



ERD 활용

도메인 간 연관관계를 한 장에서 확인할 수 있게 두고, 각 도메인 상세 페이지와 문제 해결 문서로 내려가도록 구성했습니다.

문서 허브

architecture,
domains,
troubleshooting,
refactoring,
concurrency,

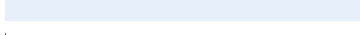
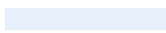
데모

백엔드 없이도 mock 계층으로 핵심 UI 흐름을 확인할 수 있는 React 데모 앱을 포함했습니다.

Case 1. Care 요청 목록 N+1

페이징된 펫케어 요청 목록에서 CareRequest의 applications를 DTO 변환 중 lazy load하면서 요청 수만큼 careapplication 쿼리가 반복되는 문제가 있었습니다. Fetch Join과 BatchSize/배치 조회 관점으로 목록 조회 경로를 재정리했습니다.

Before / After

Query		2400
		5
Time ms		1084
		66
Memory MB		21
		6

원인

페이징 쿼리에서 applications fetch가 빠졌고, Converter가 applicationCount와 리스트를 만들며 lazy load를 반복했습니다.

해결

필요한 연관 데이터를 한 번에 가져오거나 BatchSize로 IN 절 조회가 일어나도록 조정했습니다. DTO 변환은 메모리 매핑으로 제한했습니다.

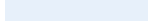
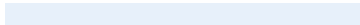
결과

1004개 데이터 기준 쿼리 수 2400개에서 4-5개, 실행 시간 1084ms에서 66ms, 메모리 21MB에서 6MB로 감소했습니다.

Case 2. Board 목록 조회 최적화

게시글 목록에서 작성자, 반응, 댓글/첨부 정보가 게시글마다 반복 조회되며 쿼리 수가 선형 증가했습니다. 게시글 ID 목록을 먼저 모으고 반응/파일/댓글 카운트를 그룹 조회한 뒤 Map으로 합치는 방식으로 바꿨습니다.

Before / After

Query		301
		3
Time ms		745
		30
Memory MB		22.5
		2

패턴

IN 절 배치 조회, Fetch Join, GROUP BY count, Map 기반 DTO 조립을 조합했습니다.

관리자 경로

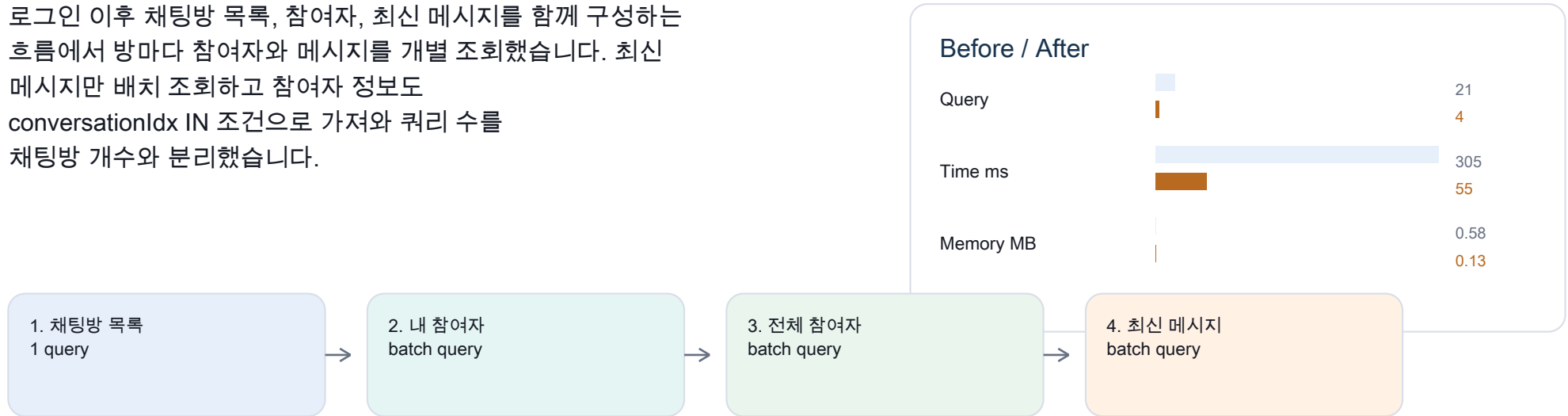
전체 게시글 로드 후 메모리 필터링하던 관리자 API를 DB 레벨 필터링과 페이징으로 전환했습니다.

효과

100개 게시글 실측 기준 301개에서 3개 쿼리, 745ms에서 30ms, 메모리 22.5MB에서 2MB로 개선했습니다.

Case 3. 로그인과 Chat 목록 N+1

로그인 이후 채팅방 목록, 참여자, 최신 메시지를 함께 구성하는 흐름에서 방마다 참여자와 메시지를 개별 조회했습니다. 최신 메시지만 배치 조회하고 참여자 정보도 conversationIdx IN 조건으로 가져와 쿼리 수를 채팅방 개수와 분리했습니다.



결과: 채팅방 10개, 참여자 30명, 메시지 200개 시나리오에서 쿼리 21개에서 4개, 실행 시간 305ms에서 55ms, 메모리 0.58MB에서 0.13MB.

동시성 제어

동시성 문제는 모두 같은 락으로 풀지 않았습니다. 비즈니스 정합성, 중복 방지, 카운터 정확성, 실패 비용을 기준으로 ~~Pessimistic Lock, DB Unique 제약조건, 읽기전 UPDATE를 구분했습니다~~

Meetup

최대 인원 초과

원자적 UPDATE 또는 비관적 락으로 참가 가능 조건을 DB에서 보장

Board

좋아요 중복

board/user unique index와 예외 처리로 중복 반응 차단

Chat

읽지 않은 메시지 Lost Update

`unread_count = unread_count + 1` 원자적 증가 쿼리

Payment/Care

거래 확정/지급 중복

트랜잭션 범위와 row lock으로 상태 전이 보호

Report/User

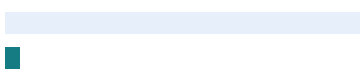
중복 신고/닉네임

DB Unique 제약조건으로 애플리케이션 체크의 빈틈 보완

위치 기반 조회 최적화

근처 모임 조회는 처음에 전체 meetup을 메모리에 로드한 뒤 Java에서 거리 계산과 필터링을 수행했습니다. 이후 DB 쿼리로 필터링을 옮겼고, 최종적으로 Bounding Box 조건을 추가해 위치 인덱스를 활용했습니다.

Before / Bounding Box

Total ms		486 273
DB ms		241 143
Memory MB		1.48 0.21
Scanned rows		2958 117

Before

findAllNotDeleted로 전체 데이터를 가져와 Java Stream에서 날짜, 상태, 좌표, 반경 필터와 정렬을 처리했습니다.

After

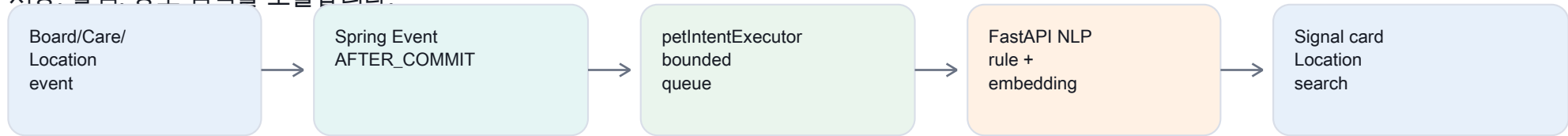
위도/경도 BETWEEN 조건으로 후보군을 줄이고 Haversine 계산을 DB에서 수행했습니다.

결과

스캔 행 수 2958개에서 117개, 메모리 1.48MB에서 0.21MB, 전체 실행 시간 486ms에서 273ms.

Recommendation / NLP

추천은 사용자의 게시글, 케어 요청, 위치 검색 이벤트를 기반으로 intent signal을 만들고 Location 검색 카테고리로 연결합니다. React는 Python 서버를 직접 호출하지 않고 Spring Boot가 이벤트 수집, 저장, 알림, 장소 검색을 조율합니다.



트래픽 제어

NLP 작업은 core 2, max 6, queue 500 전용 executor에서 처리합니다. 큐 포화 시 부가 기능 작업은 버려 핵심 요청을 보호합니다.

중복 호출 억제

Location 검색은 keyword 정규화, 길이/공백 조건, Redis setIfAbsent TTL 10분으로 반복 NLP 호출을 줄였습니다.

장애 격리

Python timeout, Redis 장애, signal 저장 실패는 원 액션을 막지 않습니다. 즉시 추천은 keyword fallback으로 기본 결과를 반환합니다.

프론트 데모와 포트폴리오 구조

이 저장소는 실행 가능한 React 포트폴리오와 Petory 백엔드 문서 자산을 함께 담습니다. 사이트는 이력서형 홈, Petory 소개, 도메인 상세, mock 기반 데모, 문서 링크 허브로 구성됩니다.

Live Demo

백엔드 없이 mock
interceptor와
mockData로 주요 화면 흐름을
브라우저에서 확인할 수 있습니다.

Domain Pages

User, Board, Care,
Missing Pet,
Location,
Recommendation,
Meetup, Chat 상세 페이지를
분리했습니다.

Docs Hub

아키텍처, 트러블슈팅, 리팩토링,
동시성, SQL migration
문서의 진입점을 제공합니다.

주요 라우트

[/](#)</portfolio/petory></demo></domains/user></domains/board></domains/care></domains/location></domains/recommendation></domains/flows></docs>

정리

이 포트폴리오가 보여주는 것

- 문제를 단순히 설명하지 않고 실제 사용자 흐름으로 재현했습니다.
- 쿼리 수, 실행 시간, 메모리 사용량을 기준으로 개선 효과를 측정했습니다.
- JPA N+1, 페이징, DTO 변환, 연관관계 조회의 비용을 도메인별로 분해했습니다.
- 동시성 문제는 락, 원자적 쿼리, Unique 제약조건을 문제 성격에 맞게 선택했습니다.
- NLP와 추천은 핵심 트랜잭션에서 분리해 실패가 원 요청에 전파되지 않도록 설계했습니다.

Contact

Email: wowong123@naver.com

GitHub: github.com/makkong1

Portfolio: makkong1.github.io/makkong1-github.io

Backend: github.com/makkong1/Petory